AIAC LAB 8.1

Name :U.Pranav Dev
Roll No : 2503A52694
Batch No : 24

Task Description #1 – Password Strength Validator

Question:
Apply AI to generate at least 3 assert test cases for is_strong_password(password) and implement the validator function.

Answer:
The Python program below implements the required function/class and verifies it using assert-based test cases.

#code

exp1.py • Python

```python
def is_strong_password(password):
    if len(password) < 8 or " " in password:
        return False

    has_upper = any(c.isupper() for c in password)
    has_lower = any(c.islower() for c in password)
    has_digit = any(c.isdigit() for c in password)
    has_special = any(not c.isalnum() for c in password)

    return has_upper and has_lower and has_digit and has_special


# Assert test cases
assert is_strong_password("Abcd@123") == True
assert is_strong_password("abcd123") == False
assert is_strong_password("Abcd@1234") == True

print("All password validation tests passed")

# User input
password = input("Enter a password: ")

if is_strong_password(password):
    print("Result: Strong Password")
else:
    print("Result: Weak Password")
```

#output

TERMINAL

```
PS C:\Users\V\OneDrive\AIAC> python exp1.py
All password validation tests passed
Enter a password: Abcd@123
Result: Strong Password
PS C:\Users\V\OneDrive\AIAC> python exp1.py
All password validation tests passed
Enter a password: abcd123
Result: Weak Password
```

Task Description #2 – Number Classification with Loops

Question:
Use AI to generate at least 3 assert test cases for a classify_number(n) function and implement it using loops.

Answer:
The Python program below implements the required function/class and verifies it using assert-based test cases.

#code

exp2.py • Python

```python
def classify_number(n):
    if not isinstance(n, (int, float)):
        return "Invalid"

    for _ in range(1):
        if n > 0:
            return "Positive"
        elif n < 0:
            return "Negative"
        else:
            return "Zero"


# Assert test cases
assert classify_number(10) == "Positive"
assert classify_number(-5) == "Negative"
assert classify_number(0) == "Zero"
assert classify_number(1) == "Positive"
assert classify_number(-1) == "Negative"
assert classify_number("10") == "Invalid"
assert classify_number(None) == "Invalid"

print("All number classification tests passed")

# User input (loop runs until the user types 'q')
while True:
    value = input("Enter a number (or 'q' to quit): ")

    if value.lower() == "q":
        break

    try:
        number = int(value)
    except ValueError:
        try:
            number = float(value)
        except ValueError:
            number = value

    print("Result:", classify_number(number))

print("Program ended")
```

#output

```
TERMINAL
PS C:\Users\V\OneDrive\AIAC> python exp2.py
All number classification tests passed
Enter a number (or 'q' to quit): 10
Result: Positive
Enter a number (or 'q' to quit): -5
Result: Negative
Enter a number (or 'q' to quit): 0
Result: Zero
Enter a number (or 'q' to quit): abc
Result: Invalid
Enter a number (or 'q' to quit): q
Program ended
```

Task Description #3 – Anagram Checker

Question:
Use AI to generate at least 3 assert test cases for is_anagram(str1, str2) and implement the function.

Answer:
The Python program below implements the required function/class and verifies it using assert-based test cases.

#code

exp3.py • Python

```python
def normalize(text):
    return "".join(c.lower() for c in text if c.isalnum())


def is_anagram(str1, str2):
    return sorted(normalize(str1)) == sorted(normalize(str2))


# Assert test cases
assert is_anagram("listen", "silent") == True
assert is_anagram("hello", "world") == False
assert is_anagram("Dormitory", "Dirty Room") == True
assert is_anagram("", "") == True
assert is_anagram("School Master!", "The Classroom") == True

print("All anagram tests passed")

# User input
str1 = input("Enter first string: ")
str2 = input("Enter second string: ")

if is_anagram(str1, str2):
    print("Result: The strings are anagrams")
else:
    print("Result: The strings are not anagrams")
```

#output

TERMINAL

```
PS C:\Users\V\OneDrive\AIAC> python exp3.py
All anagram tests passed
Enter first string: listen
Enter second string: silent
Result: The strings are anagrams
PS C:\Users\V\OneDrive\AIAC> python exp3.py
All anagram tests passed
Enter first string: hello
Enter second string: world
Result: The strings are not anagrams
```

Task Description #4 – Inventory Class

Question:
Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

Answer:
The Python program below implements the required function/class and verifies it using assert-based test cases.

#code

exp4.py • Python

```python
class Inventory:
    def __init__(self):
        self.stock = {}

    def add_item(self, name, quantity):
        self.stock[name] = self.stock.get(name, 0) + quantity

    def remove_item(self, name, quantity):
        if self.stock.get(name, 0) < quantity:
            return False
        self.stock[name] -= quantity
        return True

    def get_stock(self, name):
        return self.stock.get(name, 0)


# Assert test cases
test_inv = Inventory()
test_inv.add_item("Pen", 10)
assert test_inv.get_stock("Pen") == 10
test_inv.remove_item("Pen", 5)
assert test_inv.get_stock("Pen") == 5
test_inv.add_item("Book", 3)
assert test_inv.get_stock("Book") == 3
assert test_inv.remove_item("Book", 10) == False

print("All inventory tests passed")

# User input (menu driven)
inv = Inventory()

while True:
    print("\n1. Add item  2. Remove item  3. Check stock  4. Exit")
    choice = input("Enter your choice: ")

    if choice == "1":
        name = input("Enter item name: ")
        qty = int(input("Enter quantity: "))
        inv.add_item(name, qty)
        print("Added", qty, name)
    elif choice == "2":
        name = input("Enter item name: ")
        qty = int(input("Enter quantity: "))
        if inv.remove_item(name, qty):
            print("Removed", qty, name)
        else:
            print("Not enough stock")
    elif choice == "3":
        name = input("Enter item name: ")
        print(name, "stock =", inv.get_stock(name))
    elif choice == "4":
        print("Exiting inventory")
        break
    else:
        print("Invalid choice")
```

#output

```
TERMINAL
PS C:\Users\V\OneDrive\AIAC> python exp4.py
All inventory tests passed

1. Add item  2. Remove item  3. Check stock  4. Exit
Enter your choice: 1
Enter item name: Pen
Enter quantity: 10
Added 10 Pen

1. Add item  2. Remove item  3. Check stock  4. Exit
Enter your choice: 2
Enter item name: Pen
Enter quantity: 5
Removed 5 Pen

1. Add item  2. Remove item  3. Check stock  4. Exit
Enter your choice: 1
Enter item name: Book
Enter quantity: 3
Added 3 Book

1. Add item  2. Remove item  3. Check stock  4. Exit
Enter your choice: 3
Enter item name: Pen
Pen stock = 5

1. Add item  2. Remove item  3. Check stock  4. Exit
Enter your choice: 2
Enter item name: Pen
Enter quantity: 20
Not enough stock

1. Add item  2. Remove item  3. Check stock  4. Exit
Enter your choice: 4
Exiting inventory
```

Task Description #5 – Date Validation & Formatting

Question:
Use AI to generate at least 3 assert test cases for validate_and_format_date(date_str) to check and convert dates.

Answer:
The Python program below implements the required function/class and verifies it using assert-based test cases.

#code

exp5.py • Python

```python
from datetime import datetime


def validate_and_format_date(date_str):
    try:
        date = datetime.strptime(date_str, "%m/%d/%Y")
        return date.strftime("%Y-%m-%d")
    except ValueError:
        return "Invalid Date"


# Assert test cases
assert validate_and_format_date("10/15/2023") == "2023-10-15"
assert validate_and_format_date("02/30/2023") == "Invalid Date"
assert validate_and_format_date("01/01/2024") == "2024-01-01"
assert validate_and_format_date("13/01/2024") == "Invalid Date"

print("All date validation tests passed")

# User input
date_str = input("Enter date (MM/DD/YYYY): ")

print("Formatted Date:", validate_and_format_date(date_str))
```

#output

TERMINAL

```
PS C:\Users\V\OneDrive\AIAC> python exp5.py
All date validation tests passed
Enter date (MM/DD/YYYY): 10/15/2023
Formatted Date: 2023-10-15
PS C:\Users\V\OneDrive\AIAC> python exp5.py
All date validation tests passed
Enter date (MM/DD/YYYY): 02/30/2023
Formatted Date: Invalid Date
```

Conclusion

All five tasks were implemented and verified using assert-based test cases. The programs cover validation, edge cases, and expected outputs as required in Lab 8.